Fraud Detection Pipeline – Technical Documentation

This document provides a complete technical overview and implementation guide for the *Fraud Detection Pipeline* project. It covers the **problem statement**, system **architecture**, **tech stack and prerequisites**, detailed **installation and setup** steps, and a **code walkthrough** of each component. Follow these instructions to reproduce the project from scratch.

Project Overview

Fraud detection in financial transactions is a critical use case requiring real-time analysis of streaming data. The goal of this project is to build a **real-time fraud detection pipeline** that ingests transaction events, enriches them with additional data, applies fraud-detection rules, and produces both monitoring alerts and analytical summaries.

The key challenges addressed include:

- **Streaming ingestion** of transaction events with low latency.
- **Data enrichment** by incorporating external features (e.g. customer risk scores).
- **Multi-stage data processing** using a **medallion (Bronze-Silver-Gold)** architecture for reliability and clarity.
- **Real-time alerting** when fraud rules are triggered.
- **Batch analytics and reporting** (daily summaries of fraud activity).
- **Machine learning integration** for fraud scoring (training, tracking, and deploying an ML model).

The solution uses a **Lambda/Kappa-like** architecture (a modern Lakehouse approach). Raw events flow into a **Bronze** layer (the raw ingested data), get cleaned and enriched into a **Silver** layer (joined and cleansed data), and finally are aggregated into a **Gold** layer for reporting and machine learning.

An example of the Medallion (Bronze/Silver/Gold) design is given by Delta Lake, where Bronze holds raw data, Silver holds cleansed or joined data, and Gold holds business-

level aggregates. Delta Lake was chosen for storage because it supports ACID transactions and scalable performance.

The pipeline uses **Apache Kafka** as the ingestion source, **Apache Spark Structured Streaming** for processing, and **Delta Lake** for storage. Additional components include **Redis** (for real-time features/store), **Prometheus + Grafana** (for monitoring), **Airflow** (for orchestration of batch jobs), **dbt** (for analytical modeling), and **MLflow** (for ML experiment tracking and model registry). This combination covers end-to-end data engineering, analytics, and machine learning operations.

System Architecture

The following diagram summarizes the system architecture:

- **Kafka (Data Ingestion):** Transaction events (e.g., JSON messages) are continuously produced into a Kafka topic (`transactions`). Kafka provides a durable log of events.
- **Spark Streaming (Bronze):** A Spark Structured Streaming job reads from Kafka and writes raw events as Delta tables into the *Bronze* layer. This layer contains exactly the ingested data (timestamped, partitioned, and indexed).
- **Redis (Feature Store):** External data (e.g. customer risk scores, fraud history) is stored in Redis. The **Silver** layer job looks up features from Redis to enrich each transaction. Redis is an in-memory store ideal for real-time feature retrieval.
- **Spark Streaming (Silver):** The Silver job reads the Bronze Delta table (streaming or micro-batched) and performs cleaning and enrichment:
- Converts currencies or timestamps as needed.
- Joins or looks up external features (from Redis).
- Applies fraud-rule logic (e.g. flag transaction as fraudulent if `customer_risk_score` is low or other conditions).
- Writes the enriched results into a **Silver** Delta table.
- **Alerts:** Transactions that trigger fraud rules are written into Redis (or another alert sink) to simulate real-time alerting or further action.
- **Spark Batch (Gold):** A daily Spark job (or Airflow-scheduled job) reads the Silver Delta table and computes business-level aggregates, writing them into a **Gold** Delta table. For example, it computes total transactions and fraud counts per customer per day.

- **Airflow (Orchestration):** Airflow schedules the batch analytics job (Gold layer) on a daily basis. It ensures workflows run in order (Bronze and Silver streaming run continuously; Gold runs on a schedule).
- **dbt (Semantic Layer):** On top of the Gold data, dbt is used to build analytical models (e.g., fraud rate metrics) and run data quality tests. dbt compiles SQL models and executes them in a DuckDB instance reading from the Delta files. This provides a clear, documented, and tested transformation layer.
- **MLflow (MLOps):** We train a fraud-detection model (RandomForest) on the Silver data. MLflow is used to track experiments (metrics and parameters) and register the best model. The Silver streaming job (or a separate Spark job) then uses the registered model from MLflow to score transactions in real time. The MLflow Model Registry provides versioning, lineage, and stage management of models.
- **Prometheus & Grafana (Monitoring):** All services (data generator, Spark jobs, data stores) emit metrics via the Prometheus client library. Prometheus scrapes these metrics, and Grafana is configured to display dashboards. For example, Grafana natively supports Prometheus as a data source, allowing visualization of throughput, latency, and alert counts.
- **Other tools:** We use Docker (and Docker Compose) to simplify deployment of Kafka, Redis, Prometheus, and Grafana. Python Virtual Environments separate dependencies (especially for MLflow vs. analytics tools) to avoid conflicts.

In summary, this pipeline ingests streaming data into Kafka, processes it through streaming and batch layers in Spark + Delta Lake, enriches with Redis, and supports both BI reporting and real-time ML scoring. The architecture leverages modern data engineering best practices (Lakehouse/medallion, ELT with dbt, and MLOps with MLflow).

Tech Stack & Prerequisites

The project uses the following tools and versions:

- **Programming Language:** Python 3.12 (ensure version ≥3.8 for all libraries).
- **Apache Kafka:** version 4.x (broker, for event ingestion).
- **Apache Spark:** version 3.5.1 (with Structured Streaming support).
- **Delta Lake:** version 3.2.0 (for ACID table storage on top of Parquet).
- **Python Libraries (data engineering):**
- `pyspark==3.5.1` and `delta-spark==3.2.0` (for Spark/Delta integration).

- `kafka-python==2.3.1` (Kafka producer/consumer).
- `faker==25.9.1` (fake data generation).
- `redis==5.0.4` (Python Redis client).
- `prometheus-client==0.20.0` (exporting metrics).
- `python-dotenv==1.0.1` (loading environment variables, optional).
- **Python Libraries (analytics & ML):**
- `scikit-learn==1.5.0` (for training the ML model).
- `pandas==2.2.2` and `pyarrow==15.0.2` (data processing and Parquet I/O).
- `mlflow==2.13.0` (experiment tracking and model registry).
- **dbt (data modeling):**
- `dbt-duckdb==1.8.1` (dbt for DuckDB).
- `duckdb==1.0.0` .
- **Redis:** version 7.x (as an in-memory datastore; also used via Docker image).
- **Prometheus:** latest stable (for metrics collection).
- **Grafana:** version ~9.5 (for dashboards).
- **Airflow:** version 3.x (for orchestration). (Note: only used for the scheduled Gold job.)
- **Docker & Docker Compose:** for running Kafka, Redis, Prometheus, Grafana easily.
- **Other:** Git (for version control).

Hardware/Environment: A Linux/Mac system with enough memory to run Spark and Kafka locally (at least 8 GB RAM recommended). Java 8+ (or 17+) must be installed for Kafka/Spark.

Step-by-Step Implementation

Follow these steps to set up and run the project.

1. Clone and Prepare the Repository

1. Clone the repository (or copy project files into a directory, e.g., `fraud-pipeline`).
2. Change into the project directory and review the folder structure. A recommended tree structure is:

```
fraud-pipeline/
    data_generator/
      data_generator.py
      requirements.txt
    spark/
      bronze_etl.py
      silver_etl.py
      ml/
      ml_predictor.py
      jobs/
      gold_analytics.py
    dbt/
      fraud_analytics/
      dbt_project.yml
      models/
      staging/
        stg_silver_transactions.sql
      marts/
```

```
    fct_customer_fraud_summary.sql
    schema.yml
  ml/
    train_fraud_model.py
  docker-compose.yml
  airflow/
    dags/ (if any custom DAGs; see notes)
  monitoring/
    prometheus.yml
  .env.example
  .gitignore
  README.md
```

Adjust as needed. The `.gitignore` is configured to ignore generated data and virtual environments.

3. **Environment Variables:** Copy `.env.example` to `.env` and customize if needed (e.g., to set Kafka broker address, topic name, or other configs). Example `.env` variables:

```
KAFKA_BROKER=localhost:9092
KAFKA_TOPIC=transactions
REDIS_HOST=localhost
```

```
REDIS_PORT=6379

MLFLOW_EXPERIMENT=fraud-detection
```

These can be loaded by scripts with `python-dotenv` if used.

2. Start Infrastructure Services

We use Docker Compose to start Kafka, Zookeeper, Redis, Prometheus, and Grafana. An example `docker-compose.yml`:

```
version: '3.8'

services:

zookeeper:

image: confluentinc/cp-zookeeper:7.3.0

environment:

ZOOKEEPER_CLIENT_PORT: 2181

ports:

- "2181:2181"



kafka:

image: confluentinc/cp-kafka:7.3.0

depends_on: [zookeeper]

ports:

- "9092:9092"
```

```
environment:

KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181

KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092

KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1



redis:

image: redis:7.0

ports:

- "6379:6379"



prometheus:

image: prom/prometheus:latest

volumes:

- ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml

ports:

- "9090:9090"



grafana:

image: grafana/grafana:9.5.0

ports:
```

```
- "3000:3000"
```

- Run `docker-compose up -d`. This will launch all required services in the background.
- Kafka and Zookeeper are now running on ports 9092 and 2181. Redis is on port 6379. Prometheus is on 9090. Grafana is on 3000.
- Confirm each service is running (e.g., check `docker ps` or attempt to open `http://localhost:9090`).

3. Install Python Dependencies

Create Python virtual environments for isolation:

- **Data Engineering Environment:** In the project root, create and activate a venv (for Kafka, Spark, dbt, etc):

```
python3.12 -m venv venv

source venv/bin/activate

pip install --upgrade pip

pip install \

pyspark==3.5.1  \

delta-spark==3.2.0  \

kafka-python==2.3.1  \

faker==25.9.1  \

python-dotenv==1.0.1  \

redis==5.0.4  \
```

```
prometheus-client==0.20.0  \

dbt-duckdb==1.8.1  \

duckdb==1.0.0
```

- **ML Environment (separate):** Due to dependency conflicts (notably `protobuf` version), use a second venv for MLflow:

```
deactivate

python3.12 -m venv ml_venv

source ml_venv/bin/activate

pip install --upgrade pip

pip install \

mlflow==2.13.0  \

scikit-learn==1.5.0  \

pandas==2.2.2  \

pyarrow==15.0.2
```

Note: The MLflow environment is separate to avoid `protobuf` conflicts with dbt and Spark.

- (Optionally, you can create `requirements.txt` files under each component folder for reproducibility.)

4. Kafka Topic Setup

Ensure the Kafka topic exists for transactions:

```
# (Inside Kafka container or local Kafka setup)

$ docker exec -it <kafka_container_id> bash

$ kafka-topics --create --topic transactions --bootstrap-server localhost:9092 --partiti
```

Alternatively, use the Kafka quickstart guide:

```
bin/kafka-topics.sh --create --topic transactions --bootstrap-server localhost:9092
```

This matches the `KAFKA_TOPIC` we configured.

5. Data Generation

Navigate to `fraud-pipeline/data_generator/`. Install any dependencies there (if separated):

```
pip install -r data_generator/requirements.txt
```

Run the data generator, which produces fake transactions to Kafka:

```
python data_generator.py
```

This script uses `faker` to create realistic transaction data (with fields like `customer_id`, `amount`, etc.), and a small probability of marking a transaction as suspicious. It sends JSON messages to the Kafka `transactions` topic. (You should see console output logging each sent transaction.)

> ***Note:*** *The Kafka address (`localhost:9092`) and topic name should match your Docker Compose/KAFKA settings or your `.env`.*

6. Running Spark Streaming Jobs

Ensure your Spark environment is set up (Java installed, Spark libs installed via pip in the venv).

6.1 Bronze Layer

Run the **Bronze ETL** Spark job to continuously ingest data from Kafka into Delta:

```
cd spark

spark-submit \

--packages io.delta:delta-spark_2.12:3.2.0 \

bronze_etl.py
```

This job does the following (see code walkthrough below): connects to Kafka, reads new messages, parses JSON, and writes them to a Delta table at `delta/bronze`. It runs continuously, appending new data in micro-batches. The output is partitioned by date (e.g., `bronze_date=YYYY-MM-DD`).

You should see logs of streaming batches being processed. If permission errors occur when writing to `delta/bronze`, fix by granting access (e.g., `chmod -R 777 delta` on host).

6.2 Silver Layer

In another terminal (while Bronze is running), run the **Silver ETL** Spark job:

```
spark-submit --packages io.delta:delta-spark_2.12:3.2.0 silver_etl.py
```

This job reads the Bronze Delta table (as a stream or micro-batch), enriches each record, applies fraud rules, and writes to `delta/silver`. Key enrichment steps:

- Look up `customer_risk_score` (from Redis, using `customer_id` as key).
- Add features like `is_international`, `is_velocity_burst`, etc.
- Compute `fraud_rule_triggered` (a boolean) by evaluating rules on the features.

- If a fraud is detected, push an alert to Redis (or log it).

It uses `foreachBatch` to collect each batch of transactions, join with Redis values (for risk scores), and write out an updated Delta table. The `delta/silver` data is cleaned and indexed.

6.3 Gold Layer (Batch)

The Gold layer is batch-oriented (daily summaries). We orchestrate it via Airflow:

1. **Airflow Setup:** Install and initialize Airflow if not done. Place the DAG in `airflow/dags/`. A sample DAG (`daily_summary.py`) might look like:

```
from airflow import  DAG

from airflow.operators.python import PythonOperator

from datetime import datetime

import subprocess



def  run_spark_gold():

subprocess.run(["spark-submit", "--packages", "io.delta:delta-spark_2.12:3.2.0", "spark/j



with DAG('fraud_pipeline_gold', start_date=datetime(2026,5,1), schedule_interval='@dail

gold_task = PythonOperator(

task_id='daily_summary',

python_callable=run_spark_gold

)
```

2. **Trigger the DAG:** Start the Airflow scheduler and webserver:

```
airflow db init

airflow scheduler &

airflow webserver --port 8080
```

Then visit the Airflow UI (`http://localhost:8080`), enable and trigger the `fraud_pipeline_gold` DAG. It will run `gold_analytics.py` via `spark-submit` .

Alternatively, skip Airflow and run the job manually:

```
spark-submit --packages io.delta:delta-spark_2.12:3.2.0 spark/jobs/gold_analytics.py
```

This Spark job reads the Silver Delta table (fully up to date), computes aggregates (transactions per customer per day, fraud counts, etc.), and writes results to `delta/gold/customer_daily_summary` . It overwrites or upserts daily, forming the *Gold* analytics dataset.

7. Running dbt (Semantic Layer)

We use dbt with DuckDB to create analytic views:

1. Go to `dbt/fraud_analytics` and ensure the `profiles.yml` is configured for DuckDB (pointing to `fraud_analytics.duckdb`).
2. Run `dbt deps` if any packages (not needed here).
3. Run `dbt run` to execute models:

```
cd dbt/fraud_analytics

dbt run
```

This creates:

- A staging model `stg_silver_transactions` that selects from the Silver Parquet files.
- A fact model `fct_customer_fraud_summary` that groups by customer/day.

4. Run `dbt test` to validate (schema tests ensuring no nulls in key fields).
5. Optionally generate docs with `dbt docs generate` and `dbt docs serve` (browse on `http://localhost:8081`).

If tests fail due to leftover example models, remove the `models/example/` folder or run only specific tests. (In our case, only the tests for `fct_customer_fraud_summary` should be relevant.)

8. Training and Registering the ML Model

1. Activate the ML environment:

```
source ml_venv/bin/activate
```

2. (Optional) Start the MLflow UI to monitor training:

```
mlflow ui --backend-store-uri file:./mlflow_tracking --host 0.0.0.0 --port 5000
```

Open `http://localhost:5000` in a browser. Initially it will have no experiments.

3. In the project root, run the training script:

```
python ml/train_fraud_model.py
```

This script loads the Silver data (from Delta Parquet), splits into train/test, trains a `RandomForestClassifier`, logs metrics and parameters to MLflow, and registers the model in the MLflow Model Registry under the name `fraud_detection_model`.

4. After completion, check the MLflow UI. You should see an experiment named "fraud-detection" with a run, metrics (accuracy, F1, etc.), and a registered model version. Promote the model to "Production" stage via the UI.

5. The **Silver ETL** can now use the registry URI (`models:/fraud_detection_model/Production`) to load the latest production model for scoring.

9. Monitoring (Prometheus & Grafana)

- **Prometheus:** The `monitoring/prometheus.yml` config should scrape metrics endpoints (e.g. `pushgateway` or custom HTTP servers). In our Python code we used `prometheus_client.start_http_server(port=8000)`. Ensure Prometheus is pointed to `localhost:8000` (or the relevant port) in `prometheus.yml`.
- **Grafana:** Log into Grafana (`localhost:3000`) with default credentials. Add Prometheus (`http://prometheus:9090` or `http://localhost:9090`) as a data source. Create dashboards to visualize metrics like transactions per second, fraud alerts, etc.

Prometheus is an open-source monitoring system; Grafana provides out-of-the-box support for Prometheus. You can build dashboards (or use provided ones) to track the pipeline health.

Code Walkthrough

Below is a listing of all key code files, with explanations. The directory structure is shown above.

`data_generator/data_generator.py`

```
from faker import Faker

from kafka import KafkaProducer

import json

import time, random, uuid
```

```
# Load environment variables (optional)

# from dotenv import load_dotenv

# load_dotenv()

# BROKER = os.getenv("KAFKA_BROKER", "localhost:9092")

BROKER = "localhost:9092"

TOPIC = "transactions"



# Initialize Kafka producer and Faker

producer = KafkaProducer(

bootstrap_servers=BROKER,

value_serializer=lambda  v: json.dumps(v).encode('utf-8')

)

fake = Faker()



print(f"Starting data generator, sending to {BROKER}, topic '{TOPIC}'")

while  True:

# Generate fake transaction data

transaction = {

"transaction_id": str(uuid.uuid4()),

"customer_id": random.randint(1, 100),
```

```
"amount": round(random.uniform(5.0, 500.0), 2),

"category": random.choice(["food", "electronics", "clothing", "travel"]),

"is_international": random.choice([0, 1]),

"baseline_risk_score": random.randint(1, 10),

"previous_fraud_count": random.randint(0, 3),

"account_age_days": random.randint(1, 365),

"timestamp": int(time.time())

}

# Randomly flag some as fraud (for testing)

if random.random() < 0.02:

transaction["is_fraud"] = 1

else:

transaction["is_fraud"] = 0



# Send the transaction to Kafka

producer.send(TOPIC, transaction)

print("Sent:", transaction)

time.sleep(random.uniform(0.2, 0.5))
```

Explanation: This script runs an infinite loop generating fake transaction events. Each event is a JSON dict with fields like `customer_id`, `amount`, etc. About 2% of events are

marked as `is_fraud=1`. It sends each event to the Kafka topic `transactions`. The `Faker` library is used for realism, and `kafka-python` for the producer. Adjust `BROKER` and `TOPIC` as needed.

spark/bronze_etl.py

```
from pyspark.sql import SparkSession

from pyspark.sql.functions import from_json, col

from pyspark.sql.types import StructType, StructField, StringType, IntegerType, DoubleT

import os



# Spark session with Delta support

spark = SparkSession.builder.appName("BronzeETL") \

.config("spark.jars.packages", "io.delta:delta-spark_2.12:3.2.0") \

.getOrCreate()

spark.sparkContext.setLogLevel("WARN")



# Define the schema of incoming JSON data

schema = StructType([

StructField("transaction_id", StringType(), False),

StructField("customer_id", IntegerType(), False),

StructField("amount", DoubleType(), False),

StructField("category", StringType(), True),
```

```
StructField("is_international", IntegerType(), True),

StructField("baseline_risk_score", IntegerType(), True),

StructField("previous_fraud_count", IntegerType(), True),

StructField("account_age_days", IntegerType(), True),

StructField("timestamp", LongType(), False),

StructField("is_fraud", IntegerType(), True)

])



# Read from Kafka topic "transactions"

kafka_df = spark.readStream.format("kafka") \

.option("kafka.bootstrap.servers", os.getenv("KAFKA_BROKER", "localhost:9092")) \

.option("subscribe", os.getenv("KAFKA_TOPIC", "transactions")) \

.option("startingOffsets", "earliest") \

.load()



# Parse the JSON messages from Kafka

value_df = kafka_df.select(from_json(col("value").cast("string"), schema).alias("data"))

transactions_df = value_df.select("data.*")



# Add a date partition for organization (YYYY-MM-DD)
```

```
transactions_df = transactions_df \

.withColumn("bronze_date", col("timestamp").cast("timestamp").cast("date"))



# Write to Delta Lake (Bronze layer)

checkpoint_path = "spark/checkpoints/bronze"

delta_path = "delta/bronze"

query = transactions_df.writeStream \

.format("delta") \

.option("checkpointLocation", checkpoint_path) \

.outputMode("append") \

.partitionBy("bronze_date") \

.start(delta_path)



print("Started Bronze ETL stream.")

query.awaitTermination()
```

Explanation: This Spark job continuously reads from Kafka and writes raw transactions to a Delta table in `delta/bronze`. The `value` field from Kafka is parsed as JSON using the defined `schema`. We also derive a `bronze_date` column (the date of the transaction) to partition the data by day. The stream is checkpointed to enable fault-tolerance. The `outputMode("append")` ensures each micro-batch appends new rows to the Bronze table.

spark/silver_etl.py

```
from pyspark.sql import SparkSession

from pyspark.sql.functions import col, lit, when

import redis

import pandas as pd



# Spark session (with Delta)

spark = SparkSession.builder.appName("SilverETL") \

.config("spark.jars.packages", "io.delta:delta-spark_2.12:3.2.0") \

.getOrCreate()

spark.sparkContext.setLogLevel("WARN")



# Initialize Redis client for feature lookup and alerts

redis_client = redis.Redis(

host=os.getenv("REDIS_HOST", "localhost"),

port=int(os.getenv("REDIS_PORT", "6379")),

db=0

)



# Function to process each micro-batch

def  process_batch(batch_df, batch_id):
```

```
if batch_df.count() == 0:

return

# Collect batch to Pandas for row-by-row processing

pdf = batch_df.toPandas()

enriched = []

for _, row in pdf.iterrows():

cust_id = row["customer_id"]

# Lookup customer risk score from Redis (simulate feature store)

risk_score = redis_client.get(f"risk_score:{cust_id}")

risk_score = int(risk_score) if risk_score else  0

# Fraud rule: e.g., high international OR low risk score

is_fraud = (risk_score < 3  or row["is_international"] == 1)

# Build enriched record

enriched.append({

"transaction_id": row["transaction_id"],

"customer_id": cust_id,

"amount": row["amount"],

"category": row["category"],

"is_international": row["is_international"],

"customer_risk_score": risk_score,
```

```
"baseline_risk_score": row["baseline_risk_score"],

"previous_fraud_count": row["previous_fraud_count"],

"account_age_days": row["account_age_days"],

"is_velocity_burst": int(row["amount"] > 1000.0),

"fraud_rule_triggered": int(is_fraud),

"timestamp": row["timestamp"],

"bronze_date": row["bronze_date"]

})

# If fraud detected, push alert to Redis list (or other sink)

if is_fraud:

alert = json.dumps({

"customer_id": cust_id,

"transaction_id": row["transaction_id"],

"amount": row["amount"],

"timestamp": row["timestamp"]

})

redis_client.rpush("fraud_alerts", alert)

if  not enriched:

return

# Write enriched batch to Silver Delta table
```

```
enriched_df = spark.createDataFrame(pd.DataFrame(enriched))

enriched_df.write \

.format("delta") \

.mode("append") \

.option("mergeSchema", "true") \

.save("delta/silver")



# Read Bronze Delta as a streaming source

bronze_stream = spark.readStream.format("delta").load("delta/bronze")

silver_query = bronze_stream.writeStream \

.foreachBatch(process_batch) \

.start()



print("Started Silver ETL stream.")

silver_query.awaitTermination()
```

Explanation: This job reads from the Bronze Delta table (as a stream). For each micro-batch (`process_batch`), it converts the Spark DataFrame to Pandas rows. It then:

- Retrieves `customer_risk_score` from Redis for each `customer_id` .
- Computes a simple fraud rule (`is_fraud` if risk score is low or transaction is international).

- Appends the enriched data (including new features like `is_velocity_burst`) into the Silver Delta table.
- Sends fraud alerts by pushing JSON strings into a Redis list `fraud_alerts` if `fraud_rule_triggered` is true.

This approach (using `foreachBatch`) allows arbitrary Python logic (and external calls to Redis) on each batch of Spark data. Finally, it writes the enriched batch back to the Delta Silver table. The table grows over time with cleansed, feature-enhanced transactions.

spark/jobs/gold_analytics.py

```
from pyspark.sql import SparkSession

from pyspark.sql.functions import count, sum, avg, max  as spark_max

import os



# Spark session (with Delta)

spark = SparkSession.builder.appName("GoldAnalytics") \

.config("spark.jars.packages", "io.delta:delta-spark_2.12:3.2.0") \

.getOrCreate()



# Read the Silver table (all data available so far)

silver_df = spark.read.format("delta").load("delta/silver")



# Aggregate: daily summary of transactions per customer

gold_df = silver_df.groupBy("bronze_date", "customer_id").agg(
```

```
count("*").alias("total_transactions"),

sum("fraud_rule_triggered").alias("fraud_transactions"),

avg("amount").alias("avg_transaction_amount"),

sum("amount").alias("total_transaction_amount"),

avg("customer_risk_score").alias("avg_customer_risk_score"),

spark_max("customer_risk_score").alias("max_customer_risk_score")

)



# Write to Gold Delta table

gold_df.write \

.format("delta") \

.mode("overwrite") \

.save("delta/gold/customer_daily_summary")



print("Gold table updated.")
```

Explanation: This batch job processes the entire Silver dataset (or new data) to produce daily aggregates. We group by the transaction date (`bronze_date`) and `customer_id` , computing metrics like total transactions, fraud counts, average and total amount, and customer risk score stats. The result is saved to a Gold Delta table (overwritten each run). This Gold table (`delta/gold/customer_daily_summary`) can be used for reporting or feeding downstream analytics.

spark/ml/ml_predictor.py

```
from  __future__  import annotations

import mlflow.pyfunc

import pandas as pd



# Load the latest production model from MLflow

MODEL_URI = "models:/fraud_detection_model/Production"

model = mlflow.pyfunc.load_model(MODEL_URI)



# Define the features our model expects (in order)

FEATURE_COLUMNS = [

"amount",

"is_international",

"is_velocity_burst",

"baseline_risk_score",

"previous_fraud_count",

"account_age_days"

]



def  predict_transaction(features: dict) -> int:

"""
```

```
Perform fraud prediction for a single transaction.

`features` is a dict with keys matching FEATURE_COLUMNS.

Returns 1 if fraud predicted, else 0.

"""

# Create DataFrame for the model

input_df = pd.DataFrame([[features[col] for col in  FEATURE_COLUMNS]], columns=FEAT

prediction = model.predict(input_df)

return  int(prediction[0])
```

Explanation: This utility loads the **production-ready** ML model from the MLflow Model Registry (using the URI `"models:/<model_name>/Production"` 17†L121-L129). The `predict_transaction` function takes a transaction's feature dict, builds a Pandas DataFrame, and calls the model's `predict`. The returned label is converted to an integer. In a real system, you would integrate this into the Silver stream (e.g. using a Pandas UDF or broadcasting the model) to add an `ml_prediction` field.

dbt/fraud_analytics/dbt_project.yml

```
name: 'fraud_analytics'

version: '1.0'

config-version: 2



profile: 'fraud_analytics'

target-path: 'target'
```

```
clean-targets:

- 'target'

- 'dbt_packages'
```

Explanation: Basic dbt project configuration. It references the `fraud_analytics` profile, which should use DuckDB. This file is auto-generated by `dbt init`.

dbt/fraud_analytics/models/staging/stg_silver_transactions.sql

```
-- This staging model reads the Silver Delta table via DuckDB's Parquet reader.

select *

from read_parquet('../../delta/silver/**/*.parquet')
```

Explanation: A simple model that loads all Parquet files from the `delta/silver` directory into DuckDB. This makes the streamed data queryable in dbt. The double-star (`**`) is a glob pattern to include partition subfolders.

dbt/fraud_analytics/models/marts/fct_customer_fraud_summary.sql

```
with base as (

select * from {{ ref('stg_silver_transactions') }}

)

select

bronze_date,

customer_id,
```

```
count(*) as total_transactions,

sum(

case  when fraud_rule_triggered = 1  then  1  else  0  end

) as fraud_transactions,

round(avg(amount), 2) as avg_transaction_amount,

round(sum(amount), 2) as total_transaction_amount,

round(avg(customer_risk_score), 4) as avg_customer_risk_score,

max(customer_risk_score) as max_customer_risk_score

from base

group by bronze_date, customer_id
```

Explanation: A dbt model (SQL) that builds on the staging data (`stg_silver_transactions`) to compute the same aggregates as in the Gold ETL. It groups by date and customer, counts transactions, sums up fraud flags, and computes averages. This model can be materialized as a table or view in DuckDB.

dbt/fraud_analytics/models/marts/schema.yml

```
version: 2



models:

- name: fct_customer_fraud_summary

columns:

- name: customer_id
```

```
tests:

- not_null

- name: total_transactions

tests:

- not_null
```

Explanation: Schema tests for the `fct_customer_fraud_summary` model. We test that `customer_id` and `total_transactions` are never null (simple data quality checks). Running `dbt test` will execute these.

ml/train_fraud_model.py

```
from __future__ import annotations

import mlflow

import mlflow.sklearn

import pandas as pd

from sklearn.ensemble import RandomForestClassifier

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score



MLFLOW_EXPERIMENT = "fraud-detection"



def main():
```

```
# Load Silver data from Delta (DuckDB can read the Parquet files directly)

df = pd.read_parquet("delta/silver/bronze_date=*/part-*.parquet")



# Features and label

features = [

"amount", "customer_risk_score", "previous_fraud_count",

"account_age_days", "avg_transaction_amount", "is_velocity_burst"

]

X = df[features]

y = df["fraud_rule_triggered"].astype(int)



# Split into train/test

X_train, X_test, y_train, y_test = train_test_split(

X, y, test_size=0.2, random_state=42

)



# Set MLflow tracking

mlflow.set_tracking_uri("file:./mlflow_tracking")

mlflow.set_experiment(MLFLOW_EXPERIMENT)
```

```
with mlflow.start_run():

# Train model

model = RandomForestClassifier(n_estimators=100, max_depth=10, random_state=42)

model.fit(X_train, y_train)

preds = model.predict(X_test)

# Compute metrics

accuracy = accuracy_score(y_test, preds)

precision = precision_score(y_test, preds)

recall = recall_score(y_test, preds)

f1 = f1_score(y_test, preds)



# Log params and metrics

mlflow.log_param("n_estimators", 100)

mlflow.log_param("max_depth", 10)

mlflow.log_metric("accuracy", accuracy)

mlflow.log_metric("precision", precision)

mlflow.log_metric("recall", recall)

mlflow.log_metric("f1_score", f1)



# Log and register model
```

```
mlflow.sklearn.log_model(

sk_model=model,

artifact_path="fraud_model",

registered_model_name="fraud_detection_model"

)



print(f"Accuracy: {accuracy:.4f}, F1: {f1:.4f}")



if __name__ == "__main__":

main()
```

Explanation: This Python script trains a Random Forest model on the Silver data. It logs all relevant information to MLflow:

- Sets the tracking URI to `mlflow_tracking` (a local folder) so MLflow UI and the script use the same store.
- Creates or uses the experiment `"fraud-detection"`.
- Logs parameters (`n_estimators`, `max_depth`) and metrics (`accuracy`, `precision`, etc.).
- Registers the trained model under the name `fraud_detection_model` in the MLflow Model Registry.

Running this script results in an entry in MLflow UI and a registered model version. The output prints the model's performance.

`airflow/dags/fraud_pipeline_dag.py` (example)

```
from airflow import  DAG

from airflow.operators.python import PythonOperator

from datetime import datetime

import subprocess



def  run_spark(task_script):

cmd = ["spark-submit", "--packages", "io.delta:delta-spark_2.12:3.2.0", task_script]

subprocess.run(cmd, check=True)



with DAG('fraud_pipeline', start_date=datetime(2026,5,1), schedule_interval='@daily', ca

run_gold = PythonOperator(

task_id='compute_daily_summary',

python_callable=run_spark,

op_kwargs={'task_script': '/opt/project/spark/jobs/gold_analytics.py'}

)
```

Explanation: This example Airflow DAG defines a single task to run the `gold_analytics.py` Spark job daily. You may add similar tasks for Bronze/Silver if needed. The `run_spark` function simply calls `spark-submit` with the given script. Adjust paths and task dependencies as required.

Usage Instructions

1. **Start services:** Ensure Docker services are running (`docker-compose up`), and Python virtualenvs are activated as appropriate.
2. **Produce data:** Run the data generator (`python data_generator.py`) to stream transactions into Kafka.
3. **Run ETL streams:** In separate terminals, run the Spark jobs:

- `spark-submit bronze_etl.py` (continuously ingest to Bronze)
- `spark-submit silver_etl.py` (continuously process to Silver)

4. **Monitor:** Check Grafana dashboards and Redis for alerts. Grafana (port 3000) should be connected to Prometheus (port 9090). Prometheus will show metrics scraped from our Python/Spark jobs.
5. **Schedule Gold:** Either trigger the Airflow DAG or manually run `spark-submit spark/jobs/gold_analytics.py` daily. This updates the Gold summaries.
6. **Run dbt:** Navigate to `dbt/fraud_analytics` and execute:

```
dbt run

dbt test
```

to build the analytical model and validate it. Generate docs if desired.

7. **Train ML model:** Activate the MLflow venv and run `python ml/train_fraud_model.py` . Use the MLflow UI (`http://localhost:5000`) to inspect the experiment and register the model.
8. **Real-time scoring:** The Silver stream job can load the registered model (from `models:/fraud_detection_model/Production`) and append predictions to each transaction (not shown explicitly, but the `ml_predictor.py` utility demonstrates how).

Expect the output to include:

- Streams of logs showing Spark micro-batches writing to Delta.
- Fraud alerts in Redis (check by `redis-cli LRANGE fraud_alerts 0 -1`).
- A populated Delta table structure under `delta/bronze` , `delta/silver` , `delta/gold` .

- Dashboards in Grafana showing metrics (you may need to import or create dashboards).
- dbt logs showing model creation.
- MLflow UI showing experiment metrics and registered model.

Troubleshooting

Below are common issues and resolutions encountered during development:

- **Kafka connection refused:** Ensure Kafka/Zookeeper are running. Check Docker logs (`docker-compose ps`) and verify broker at `localhost:9092` . In Docker Compose, we set `ADVERTISED_LISTENERS=PLAINTEXT://localhost:9092` . If running Kafka manually, start Zookeeper and then `bin/kafka-server-start.sh config/server.properties` .
- **Spark Delta write permission errors:** If Spark fails to write to `delta/` , change directory permissions: `chmod -R 777 delta` . This was needed because container user did not have write access.
- **dbt test failures (example models):** The default dbt project includes sample models (`models/example`). Remove this folder (or `schema.yml`) so that only your custom models are tested.
- **dbt docs port in use:** If `dbt docs serve` fails with address in use, specify another port: `dbt docs serve --port 8081` .
- **MLflow dependency conflicts:** If pip cannot install MLflow due to `pyarrow` / `protobuf` conflicts, use separate venvs as above. For example, `dbt-core` requires `protobuf<7.0,>=6.0` while the latest MLflow uses a higher version, so separate environments avoid this issue.
- **MLflow UI shows no runs:** Ensure the training script and MLflow UI use the same tracking URI. We set `mlflow.set_tracking_uri("file:./mlflow_tracking")` in the script and started `mlflow ui --backend-store-uri file:./mlflow_tracking` .
- **Nested Git issues:** If you initialized a Git repo inside the project subfolder by mistake, remove the inner `.git` directory and re-add files so that the top-level repo tracks everything correctly.
- **Spark Structured Streaming offset error:** If Spark complains about missing checkpoints or offset resets, either delete the checkpoint directory

(`spark/checkpoints/bronze`) to restart streaming, or properly configure `startingOffsets` .

- **Prometheus not scraping:** Verify that your Python jobs call `start_http_server(port)` from `prometheus_client` and that `prometheus.yml` includes that endpoint in `scrape_configs` .
- **Model not found in MLflow:** After registering, ensure to load with the correct URI. We used `models:/fraud_detection_model/Production` . In MLflow UI, make sure the model is in the Production stage or refer to version by number (e.g. `models:/fraud_detection_model/1`).
- **Data not updating:** If the Gold job produces empty results, check that the Silver Delta table has data. You can run a Spark shell or use DuckDB to query the Parquet files manually.

Each of these issues was encountered and resolved during development. Following the setup steps carefully should avoid most problems.

Conclusion

This documentation covers all aspects of building the Fraud Detection Pipeline: from data ingestion to real-time processing, analytics, and ML integration. The project demonstrates a **Lakehouse** style architecture with modern tooling (Delta Lake, dbt, MLflow) and should be reproducible by following the steps above. The use of citations supports design decisions (Medallion architecture and MLflow model management).

Developers can extend this pipeline by adding further business logic, more robust alerting, or deploying to cloud environments. The modular design ensures each component (Kafka, Spark, dbt, MLflow, etc.) can be scaled or replaced as needed.